Ordenamiento de un dataset grande en C++

Nash Díaz, Valentina Escobar y Mariana Escobar · 1 de noviembre de 2026

1. Introducción

En esta práctica ordenamos alfabéticamente 100.000 palabras en inglés con tres estrategias que implementamos desde cero en C++17, cada una apoyada en una estructura de datos distinta. Nuestro objetivo no era solo que funcionaran, sino entender por qué, si en teoría las tres cuestan O(n log n), en la práctica no tardan lo mismo.

El resultado más claro fue que **QuickSort sobre un arreglo dinámico ganó con holgura**: 32,5 ms en promedio y prácticamente sin memoria extra. Lo que no esperábamos es que HeapSort (49,0 ms) y el árbol AVL (47,7 ms) quedaran casi empatados. Llegan ahí por caminos muy distintos: el AVL hace la mitad de las comparaciones que HeapSort, pero gasta casi tres veces más memoria y es el que más varía de una repetición a otra.

Estrategia	Estructura de datos	Archivo
A) QuickSort	Arreglo dinámico `std::vector<std::string>`	`src/quicksort.hpp`
B) HeapSort	Montículo binario (min-heap) guardado en un arreglo	`src/heapsort.hpp`
C) Árbol AVL + recorrido inorden	Árbol binario de búsqueda autobalanceado	`src/avl.hpp`
Extra: HeapSort in situ	Max-heap dentro del mismo arreglo	`src/heapsort.hpp`

No usamos `std::sort`, `std::priority_queue`, `std::make_heap`, `std::set` ni `std::map`: todo lo que ordena en este proyecto está escrito a mano.

2. El dataset

Trabajamos con `dataset.txt`: 100.000 palabras distintas, en minúsculas y en orden aleatorio, tomadas de `words_alpha.txt`, que tiene 370.105 palabras.

Un detalle importante es que `words_alpha.txt` viene ordenado alfabéticamente. Si lo hubiéramos usado tal cual, no estaríamos midiendo el caso típico sino el de una entrada ya ordenada, que para algunos algoritmos es el mejor caso y para otros el peor. Por eso el dataset tiene que ser una muestra aleatoria.

Para que ese paso se pueda repetir, escribimos un generador (`tools/generar_dataset.cpp`) con el algoritmo de **Fisher-Yates parcial**. Para i = 0 … 99.999 elige una posición j al azar entre i y el final de la lista e intercambia las palabras de i y j. Al terminar, las primeras 100.000 posiciones forman una muestra uniforme y sin repetidos. Usamos una semilla fija (2026) y nuestra propia función para sacar números en un rango, porque `std::uniform_int_distribution` puede dar resultados distintos según el compilador. Así, la misma semilla produce el mismo archivo en cualquier sistema.

Al leer el archivo apareció un detalle que nos pudo haber costado caro: las líneas terminan en `\r\n` (formato de Windows). En Linux o macOS cada palabra quedaría con un carácter invisible al final y las comparaciones saldrían mal, así que el lector elimina ese `\r`.

Otro dato que resultó importante para la memoria: 3.030 de las 100.000 palabras (3 %) tienen más de 15 letras. En la sección 4 explicamos por qué ese número importa.

3. Cómo implementamos cada estrategia

A) QuickSort sobre `std::vector<std::string>`

QuickSort es divide y vencerás: se escoge un pivote, se reparte el arreglo entre los elementos «≤ pivote» y «≥ pivote», y se repite con cada lado. Todo ocurre dentro del mismo arreglo, sin memoria adicional. La versión de libro funciona, pero tiene trampas conocidas, así que tomamos tres decisiones:

1. **Pivote por mediana de tres.** Ordenamos `v[lo]`, `v[medio]` y `v[hi]` y tomamos como pivote el del medio. Si el pivote fuera siempre el primer elemento, una entrada ya ordenada partiría cada rango en 1 y n−1 elementos y el algoritmo se volvería $O(n^2)$. En la sección 7.3 lo provocamos a propósito para verlo.
2. **Partición de Hoare.** Dos índices avanzan desde los extremos y solo intercambian cuando ambos encuentran un elemento en el lado equivocado. Hace unas tres veces menos intercambios que la partición de Lomuto. Además, como ambos índices se detienen en elementos iguales al pivote, una entrada con muchos repetidos se sigue partiendo por la mitad.
3. **Recursión sobre la parte pequeña y ciclo sobre la grande.** La parte pequeña tiene como mucho n/2 elementos, así que la pila nunca pasa de $\log_2 n$ niveles, ni siquiera en el peor caso.

```
quickSort(v, lo, hi):
    mientras lo < hi:
        ordenar v[lo], v[medio], v[hi];  pivote ← v[medio]
        i ← lo − 1;  j ← hi + 1
        repetir:
            avanzar i mientras v[i] < pivote
            retroceder j mientras v[j] > pivote
            si i ≥ j: salir
            intercambiar v[i], v[j]
        si (j − lo) < (hi − j): quickSort(v, lo, j);  lo ← j + 1
        si no:                  quickSort(v, j + 1, hi);  hi ← j
```

B) HeapSort con un montículo binario

Implementamos la clase `MonticuloBinarioMin`: un árbol binario completo guardado en un arreglo, sin punteros. Los hijos de la posición i están en 2i+1 y 2i+2, y su padre en (i−1)/2. Cada padre es ≤ que sus hijos, así que la palabra menor siempre está en la raíz. Tiene `insertar` (flotar, O(log n)), `extraerMin` (hundir, O(log n)), `minimo` (O(1)) y la construcción de Floyd (O(n)).

El HeapSort sigue exactamente lo que pide el enunciado: construimos el heap con las 100.000 palabras y extraemos el mínimo 100.000 veces hacia una lista de salida. El heap reutiliza el arreglo de entrada (se mueve, no se copia), así que la única memoria extra es esa lista.

Como extra hicimos también el **HeapSort in situ** clásico, con un max-heap dentro del mismo arreglo: el máximo se cambia con la última posición y el heap se encoge en uno. Tiene el mismo O(n log n) pero no necesita memoria extra, y nos sirvió para separar el costo del algoritmo del costo de la lista.

C) Árbol AVL + recorrido inorden

Un AVL es un árbol binario de búsqueda donde, en cada nodo, la diferencia de alturas entre los subárboles izquierdo y derecho es −1, 0 o +1. Con eso la altura nunca pasa de unos $1{,}44 \cdot \log_2 n$. Cada nodo guarda la palabra, su altura, un contador de repeticiones y dos punteros.

- **Inserción recursiva:** baja como en cualquier BST y, al regresar, actualiza la altura de cada ancestro. Si un factor de balance llega a ±2, aplica una de las cuatro rotaciones: Izq-Izq (simple a la derecha), Der-Der (simple a la izquierda), Izq-Der o Der-Izq (dobles).
- **Repetidos:** si la palabra ya existe, solo aumenta el contador del nodo y la forma del árbol no cambia.
- **Orden final:** el recorrido inorden (izquierda, raíz, derecha) visita las claves en orden ascendente y las va guardando en una lista.

Escogimos AVL en lugar de Rojo-Negro porque su balance es más estricto (altura máxima $1{,}44 \cdot \log_2 n$ contra $2 \cdot \log_2 n$) y porque su regla es más fácil de comprobar: basta con comparar alturas.

Cómo sabemos que funciona

No quisimos confiar solo en que «se ve ordenado». Cada salida se revisa tres veces:

1. Que esté ordenada.
2. Que contenga exactamente las mismas palabras que la entrada. Lo comprobamos con una tabla hash, para no depender de ningún ordenamiento.
3. Que las cuatro estrategias den exactamente la misma lista.

El programa también valida la propiedad de heap y las tres propiedades del AVL (orden, balance y alturas) en la estructura que construyó. Aparte, `tests/pruebas.cpp` ejecuta **24.107 comprobaciones**: casos límite y 3.000 arreglos aleatorios con muchos repetidos, comparados contra un ordenamiento por inserción escrito a mano. Todas pasan.

4. Cómo medimos y en qué equipo

Hicimos todas las mediciones en el mismo portátil:

Componente	Detalle
Procesador	Intel Core i3-1115G4 (Tiger Lake): 2 núcleos y 4 hilos, de 3,0 a 4,1 GHz
Memoria caché	L2 de 1,25 MB por núcleo y L3 de 6 MB compartida
RAM	8 GB DDR4 a 3.200 MT/s
Sistema y compilador	Windows 11, g++ 13.1 (MinGW) con `-O2 -static`

Los tamaños de caché parecen un detalle técnico, pero terminaron explicando buena parte de los resultados (sección 7.1).

Tiempo. Usamos `std::chrono::steady_clock`, que es monótono y no cambia si se ajusta la hora del sistema. Cada estrategia corre 5 veces, siempre sobre una copia nueva del dataset hecha fuera del cronómetro, y reportamos la media, la desviación estándar, el mínimo y el máximo. Medimos el proceso completo de ordenar: en HeapSort, construir el heap y extraer; en el AVL, insertar y recorrer hacia la lista. La destrucción del árbol queda por fuera, porque es limpieza y no ordenamiento.

Memoria estimada con `sizeof`. En este compilador `sizeof(std::string)` es 32 bytes. Las palabras de hasta 15 letras caben dentro de esos 32 bytes gracias a la *Small String Optimization* (SSO), y solo las 3.030 más largas piden un bloque aparte, unos 0,05 MB en total. Un nodo del AVL mide `sizeof(Nodo)` = 56 bytes: 32 del string, 16 de los dos punteros y 8 de la altura y el contador.

Estrategia	Memoria total de la estructura	Memoria extra sobre la entrada
QuickSort	n·32 + texto largo = 3,10 MB	0 (in situ) + la pila de log n niveles
HeapSort	heap n·32 + lista n·32 + texto = 6,16 MB	lista de salida = 3,05 MB
AVL	nodos n·56 + lista n·32 + 2·texto = 8,50 MB	nodos + lista + copia del texto largo = 8,44 MB
HeapSort in situ	n·32 + texto = 3,10 MB	0

Memoria medida. Queríamos comprobar que esas cuentas fueran ciertas y no solo teóricas, así que `src/memoria.cpp` reemplaza los operadores globales `new` y `delete`. Cada reserva guarda su tamaño en una pequeña cabecera y lo suma a un contador que registra el pico; cada liberación lo resta. Con eso obtenemos cuántos bytes pidió cada algoritmo por encima de su entrada. No incluye el sobrecosto interno del asignador (unos 8 a 16 bytes por bloque, entre 0,8 y 1,5 MB para los 100.000 nodos del AVL), ni los 16 bytes de cabecera que el propio medidor agrega a cada bloque (otros 1,5 MB).

Contadores. El programa cuenta comparaciones, intercambios, rotaciones, profundidad de recursión y alturas. Estos números no dependen del computador ni de qué tan ocupado esté, y eso los volvió muy útiles, como contamos en la siguiente sección.

5. Lo que salió mal antes de medir bien

Las primeras veces que corrimos el programa no obtuvimos los resultados que esperábamos, y vale la pena contarlo porque de ahí salieron dos lecciones.

El programa se cerraba sin decir nada

La primera vez que compilamos en Windows, las pruebas imprimían «Casos limite...» y se cerraban, sin ningún mensaje de error. El problema no estaba en los algoritmos sino en el medidor de memoria. Con MinGW, la biblioteca estándar de C++ vive en una DLL aparte que no ve nuestra versión de `new` y `delete`. Entonces parte de la memoria se pedía con el `new` de la DLL y se liberaba con el nuestro, o al revés, y el programa se caía. La solución fue compilar con `-static`, para que todo quede dentro del mismo ejecutable. Desde ahí pasaron las 24.107 pruebas. Lo dejamos anotado en el README para que nadie más se tropiece con lo mismo.

La lección: **medir también es código**, y puede romper el programa que se quiere medir.

Los tiempos salían inflados y muy dispersos

La primera corrida completa la hicimos con el portátil sin cargador y con varios programas abiertos. Al repetirla conectado, en modo de máximo rendimiento y con todo lo demás cerrado, los tiempos cambiaron muchísimo:

Estrategia	Sin cargador (ms)	Con cargador y máximo rendimiento (ms)	Diferencia
QuickSort	46,20 ± 9,09	32,53 ± 0,22	+42 %
HeapSort	82,60 ± 10,92	48,99 ± 0,33	+69 %
Árbol AVL	84,91 ± 15,27	47,72 ± 4,96	+78 %
HeapSort in situ	100,75 ± 26,61	51,70 ± 2,00	+95 %

Con batería, Windows suele bajar la frecuencia del procesador para ahorrar energía, y los demás programas competían por el mismo procesador. Todo salía más lento y, peor aún, muy disperso: en la corrida sin

cargador, QuickSort variaba 9 ms entre repeticiones, frente a 0,22 ms en la limpia. Incluso el orden entre HeapSort y el AVL se invirtió de una corrida a la otra.

Lo que nos dio confianza fue que **los contadores salieron idénticos en las dos corridas**: 2.383.082 comparaciones en QuickSort, 3.019.694 en HeapSort, 46.486 rotaciones en el AVL. Los algoritmos hicieron exactamente el mismo trabajo; lo único que cambió fue el computador. De ahí sacamos dos reglas para el resto del trabajo: no sacar conclusiones de una diferencia de tiempo menor que la desviación estándar, y usar el mínimo de las repeticiones en el experimento de escalabilidad, porque el ruido externo solo puede sumar tiempo.

Todos los resultados de aquí en adelante son de la corrida limpia.

6. Resultados con n = 100.000

QuickSort fue 1,5 veces más rápido que las otras dos estrategias. Las cuatro produjeron exactamente la misma lista, ordenada y completa.

Estrategia	Tiempo medio (ms)	Desv. (ms)	Memoria extra estimada	Memoria extra medida	Comparaciones	Comparaciones / ($n \cdot \log_2 n$)
QuickSort	32,53	0,22	≈ 0 MB	30 bytes	2.383.082	1,43
Árbol AVL	47,72	4,96	8,44 MB	8,44 MB (8.854.576 B)	1.545.925	0,93
HeapSort	48,99	0,33	3,05 MB	3,05 MB (3.200.000 B)	3.019.694	1,82
HeapSort in situ (extra)	51,70	2,00	0 MB	0 bytes	3.019.433	1,82

AVL contra HeapSort: un empate. Por la media, el AVL quedó 1,27 ms por delante, pero esa diferencia es menor que su propia desviación estándar (4,96 ms). Siguiendo la regla de la sección 5, no podemos decir que uno sea más rápido que el otro. Lo que sí se ve es que se comportan distinto: HeapSort es muy estable (de 48,64 a 49,33 ms), mientras que el AVL fue de 44,47 a 56,50 ms. Creemos que esa variación viene de las 100.000 llamadas a `new` del AVL, una por nodo: el asignador de memoria no siempre tarda lo mismo.

Lo que muestran los contadores de cada estrategia:

- **QuickSort:** 418.889 intercambios en 99.999 rondas de partición. La recursión llegó a 13 niveles, por debajo de $\log_2 100.000 \approx 16,6$. Los 30 bytes medidos son la copia del pivote cuando este es una palabra larga que no cabe en el SSO; solo existe una copia a la vez.
- **HeapSort:** construir el heap tomó 3,11 ms y extraer los 100.000 mínimos 45,88 ms, así que el 94 % del tiempo se va en las extracciones. La construcción hizo 187.865 comparaciones, menos que $2n = 200.000$, lo que confirma que Floyd es O(n). El heap tiene altura 16.
- **AVL:** insertar tomó 42,15 ms y el recorrido inorden 5,57 ms. La altura final fue 20, entre la mínima posible (17) y la cota AVL ($1,44 \cdot \log_2 n \approx 23,9$). Hubo 23.309 rotaciones simples y 23.177 dobles, más o menos una por cada 2,2 inserciones.
- **Memoria:** la medida coincidió byte a byte con la estimada en HeapSort, el AVL y HeapSort in situ. En QuickSort la única diferencia son esos 30 bytes del pivote. Nos gustó confirmar que las cuentas con `sizeof` no eran solo teoría.

Los datos completos están en `results/resultados.csv` y `results/salida_consola.txt`.

7. Experimentos adicionales

7.1 ¿El tiempo crece de verdad como n log n?

Si un algoritmo es Θ(n log n), su tiempo dividido entre $n \cdot \log_2 n$ debería mantenerse constante al crecer n. Para comprobarlo ordenamos los primeros 10.000, 20.000, … 100.000 palabras del dataset, tomando en cada punto el mínimo de 5 repeticiones.

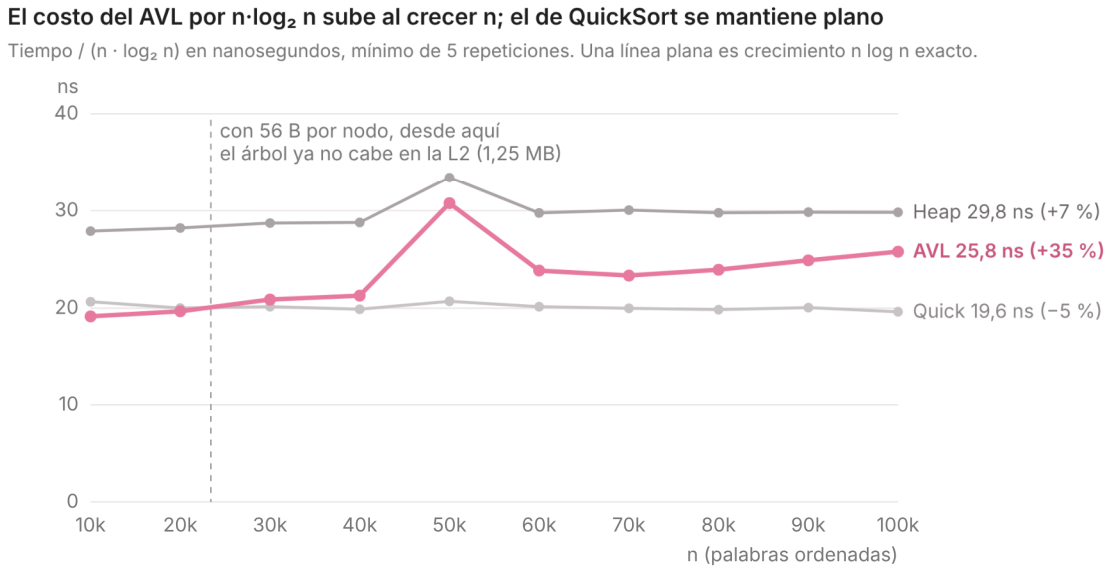


Figura 1. Fuente: `results/escalabilidad.csv`, prefijos del dataset de 10.000 a 100.000 palabras. Línea punteada: 1,25 MB de L2 ÷ 56 bytes por nodo ≈ 23.400 nodos (cota superior: cada bloque ocupa algo más).

QuickSort se mantiene plano, alrededor de 20 ns: crece exactamente como n log n. HeapSort sube un poco (+7 %) y el AVL bastante más (+35 %), aunque los tres tienen la misma complejidad.

Lo que más nos sorprendió fue el comienzo de la gráfica. Con 10.000 y 20.000 palabras, **el AVL fue más rápido que QuickSort por operación** (19,2 contra 20,7 ns con 10.000), lo cual tiene sentido porque hace menos comparaciones. Dejó de ganar entre las 20.000 y las 30.000 palabras, más o menos cuando el árbol creció más que la caché L2 de un núcleo del procesador. Contando solo los 56 bytes del nodo, eso pasa hacia los 23.400 nodos (la línea punteada). En realidad cada nodo ocupa algo más, porque el asignador y nuestro medidor agregan su propia cabecera a cada bloque, así que ese límite se alcanza incluso un poco antes. Desde ahí, cada salto a otro nodo puede tener que buscar el dato en la L3 o en la RAM, que son más lentas. Con 100.000 palabras los nodos ocupan 5,3 MB y, junto con la lista de salida, superan incluso los 6 MB de la L3. QuickSort, en cambio, recorre el arreglo en orden, y el procesador alcanza a traer los datos antes de necesitarlos.

El pico en 50.000 palabras aparece en HeapSort y en el AVL a pesar de usar el mínimo de 5 repeticiones. Los puntos vecinos son normales, así que no lo atribuimos al algoritmo. Lo tomamos como una interferencia del sistema durante esa medición, probablemente un cambio en la frecuencia del procesador.

7.2 ¿Importa cómo viene ordenada la entrada?

Repetimos la medición con cuatro tipos de entrada de 100.000 palabras (5 repeticiones cada una):

Entrada	QuickSort (ms)	HeapSort (ms)	AVL (ms)	Rotaciones AVL
Aleatoria	35,11	54,65	50,71	46.486
Ya ordenada	17,41	43,16	24,22	99.983
Orden inverso	17,36	43,72	25,15	99.983
Muchos repetidos (100 palabras distintas)	20,59	34,59	4,56	44

- **QuickSort** es 2 veces más rápido con entrada ordenada o inversa que con la aleatoria: la mediana de tres le da el pivote perfecto y la partición de Hoare casi no tiene que intercambiar.
- **El AVL con entrada ordenada** rota en casi cada inserción (99.983 rotaciones) y aun así es 2,1 veces más rápido que con la aleatoria. Siempre baja por el mismo camino, el borde derecho del árbol, así que esos nodos están en caché y el procesador acierta los saltos. Una rotación son solo tres cambios de punteros.
- **Con muchos repetidos el AVL arrasa:** 4,56 ms, el tiempo más bajo entre todos los ordenamientos de 100.000 palabras y 11 veces menos que con la entrada aleatoria. Solo crea 100 nodos; cada palabra repetida solo aumenta un contador.
- **HeapSort** es, en proporción, el que menos depende de la entrada (de 34,6 a 54,7 ms), porque cada extracción tiene que hundir un elemento por todo el heap sin importar cómo llegaron los datos.

Una observación honesta: con la entrada aleatoria, los tiempos de esta tabla son entre 6 y 12 % más altos que los de la sección 6, aunque es el mismo caso. Este experimento corre al final, después de varios segundos de trabajo continuo. Nuestra hipótesis es que, tras ese tiempo a máxima carga, el procesador ya no sostiene su frecuencia turbo más alta. Como las cuatro filas se midieron en las mismas condiciones, la comparación entre ellas sigue siendo válida.

7.3 ¿Qué tanto importa el pivote de QuickSort?

Para ver el peor caso con nuestros propios ojos, agregamos un modo de QuickSort que usa el primer elemento como pivote y lo comparamos con la mediana de tres (n = 20.000):

Pivote / entrada	Tiempo (ms)	Comparaciones	Profundidad de recursión
Mediana de tres / aleatoria	5,89	410.880	11
Mediana de tres / ya ordenada	2,94	367.228	15
Primer elemento / aleatoria	5,62	438.648	11
Primer elemento / ya ordenada	768,84	200.029.998	2

Con datos aleatorios casi no hay diferencia, pero con la entrada ordenada el pivote ingenuo es un desastre. Siempre es el menor del rango, así que cada partición separa un solo elemento de los n−1 restantes. El resultado son 200.029.998 comparaciones, prácticamente $n^2/2$ = 200.000.000: **unas 260 veces más lento** que la mediana de tres con la misma entrada, y eso con solo 20.000 palabras. La profundidad se queda en 2 porque la recursión siempre va a la parte pequeña (de un elemento) y la grande se procesa con el ciclo: el peor caso cuesta tiempo, pero no desborda la pila.

8. Análisis comparativo

¿Qué algoritmo tuvo mejor desempeño?

QuickSort, tanto en tiempo como en memoria: 32,5 ms y prácticamente nada de memoria extra. Encontramos tres razones en nuestras mediciones:

- **Recorre la memoria en orden.** Los dos índices de Hoare avanzan por el arreglo de forma secuencial, y el procesador puede traer los datos a la caché antes de necesitarlos.
- **Mueve poco.** Hizo 418.889 intercambios, frente a 1.474.758 de HeapSort. Intercambiar dos `std::string` es barato (se intercambian sus punteros internos), pero no es gratis.
- **No pide memoria.** Trabaja dentro del mismo arreglo. El AVL, en cambio, hace 100.000 llamadas a `new`, una por nodo.

¿Por qué la teoría a veces no coincide con la práctica?

Las tres estrategias son O(n log n) y aun así QuickSort tardó dos tercios de lo que tardaron las otras. Esta fue, para nosotros, la parte más interesante del trabajo. La notación Big O describe cómo crece el costo, no cuánto vale, y en el camino deja por fuera varias cosas:

1. **Las constantes.** HeapSort hace $1{,}82 \cdot n \cdot \log_2 n$ comparaciones (compara con los dos hijos en cada nivel al hundir), QuickSort 1,43 y el AVL 0,93. Todos son O(n log n), pero el trabajo real por elemento es distinto.
2. **La jerarquía de memoria.** Big O supone que todo acceso a memoria cuesta lo mismo, y no es así: leer de la caché L1 toma alrededor de 1 ns y de la RAM alrededor de 100 ns. El AVL hace menos comparaciones que nadie y, con pocos datos, fue el más rápido. En cuanto el árbol dejó de caber en la caché, su costo por operación subió 35 % (sección 7.1). Eso Big O no lo predice.
3. **El caso concreto.** Big O describe el peor caso o el promedio. QuickSort es $O(n^2)$ en el peor caso y aun así fue el más rápido, porque con la mediana de tres ese caso no aparece. Con un pivote ingenuo sí aparece, y cuesta 260 veces más (sección 7.3).
4. **El costo de comparar.** Cuenta cada comparación como O(1), pero comparar dos palabras cuesta O(L), con L la longitud del prefijo que comparten. En este dataset las palabras son cortas y el efecto es pequeño.
5. **El costo de pedir memoria.** El AVL paga 100.000 reservas al insertar, y creemos que de ahí viene buena parte de su variabilidad.
6. **La máquina.** El mismo código, con el mismo n, tardó entre 42 y 95 % más solo por estar desconectado del cargador (sección 5). La complejidad no cambió; el tiempo sí.

Ventajas y desventajas de cada estructura

Estructura	Ventajas	Desventajas	Conviene cuando…
Arreglo dinámico (QuickSort)	Memoria contigua, acceso O(1) por índice, ordena in situ; fue la más rápida	Peor caso $O(n^2)$ con un mal pivote; agregar un dato después obliga a desplazar O(n) elementos	Los datos llegan completos y se ordenan una sola vez

Estructura	Ventajas	Desventajas	Conviene cuando…
Montículo binario (HeapSort)	O(n log n) garantizado siempre; sin punteros; la versión in situ usa O(1) de memoria; el más estable entre repeticiones; da el mínimo en O(1)	Accesos dispersos (los hijos de i están en 2i+1 y 2i+2, cada vez más lejos); más comparaciones e intercambios; no es estable	Se necesita un tiempo garantizado con poca memoria, o solo los k menores (cola de prioridad)
Árbol AVL	O(log n) por inserción; mantiene el orden mientras llegan datos; permite buscar y borrar en O(log n); imbatible con muchos repetidos	2,7 veces la memoria de QuickSort (8,50 MB contra 3,10 MB); punteros que dispersan los datos; una reserva por nodo y más variabilidad	Los datos llegan de a uno y hay que consultarlos ordenados en todo momento, o hay muchos repetidos

En resumen: para ordenar un archivo completo una sola vez, **el arreglo con QuickSort es la mejor opción**. El AVL resuelve un problema más general, mantener un conjunto ordenado que cambia, y ese costo extra se paga aunque solo se quiera ordenar.

9. Conclusiones

1. **Para este problema, QuickSort sobre un arreglo dinámico es la estrategia adecuada.** Los datos llegan completos, se ordenan una sola vez y no hay que mantenerlos ordenados después. Fue el más rápido (32,5 ms) y no necesitó memoria adicional. Eso sí, solo es confiable con dos decisiones de diseño: el pivote por mediana de tres y la recursión sobre la parte pequeña.
2. **La misma complejidad no significa el mismo tiempo.** Las tres estrategias son O(n log n) y QuickSort fue 1,5 veces más rápido que las otras dos. Lo que decidió el resultado no fue la cantidad de comparaciones sino cómo cada estructura usa la memoria: contigua en el arreglo, dispersa en el heap y en el árbol.
3. **La estructura de datos determina la memoria más que el algoritmo.** El AVL usó 8,44 MB extra por los nodos y sus punteros; HeapSort 3,05 MB por la lista de salida, y 0 MB en su versión in situ con un tiempo similar. La memoria medida coincidió con la estimada con `sizeof`.
4. **Cada estructura tiene su lugar.** El AVL ganó cuando había pocos datos y el árbol cabía en la caché, y fue el más rápido de todos con muchos repetidos (4,56 ms). Conviene cuando los datos llegan de a uno y hay que consultarlos ordenados. El heap conviene cuando se necesita un tiempo garantizado con poca memoria, o solo los primeros k elementos.
5. **Medir bien también es parte del trabajo.** Un cargador desconectado infló los tiempos hasta 95 %, y el propio medidor de memoria hizo caer el programa hasta que compilamos con `-static`. Los contadores, que no dependen de la máquina, fueron nuestra referencia para saber qué cambiaba por el algoritmo y qué por el computador. También aprendimos a mirar la desviación estándar antes de declarar a un ganador.

10. Uso de herramientas de IA

Usamos Claude (Anthropic) como asistente para diseñar la estructura vacía del programa (que “diseño” debe tener el programa”), ayudarnos a usar los comandos correctos para ejecutar (por ejemplo agregar -static) y ayudarnos a desarrollar este informe de manera formal y bien presentada (precisamente en los gráficos y referencias). Por nuestra parte, desarrollamos, compilamos y cada una por aparte, ejecutó todo en su equipo

(al final escogimos presentar los resultados del equipo que mejor nos pareció). Diagnosticamos con su ayuda el cierre del programa en Windows y lo resolvimos compilando con `-static`. Repetimos las mediciones en condiciones controladas cuando vimos que los primeros tiempos no tenían sentido. Simplificamos y revisamos cada módulo hasta poder explicarlo. Todos los números de este informe salen de nuestras propias ejecuciones (`results/`).

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulos 6 (HeapSort) y 7 (QuickSort).
- Sedgewick, R. y Wayne, K. *Algorithms*, 4.ª ed. Addison-Wesley, 2011. Secciones 2.3 (QuickSort, mediana de tres) y 2.4 (colas de prioridad y montículos).
- Knuth, D. E. *The Art of Computer Programming*, vol. 3: *Sorting and Searching*, 2.ª ed. Addison-Wesley, 1998. Sección 6.2.3 (árboles balanceados, cota $1{,}44 \cdot \log_2 n$).
- Hoare, C. A. R. "Quicksort". *The Computer Journal*, 5(1), 10–16, 1962.
- Adelson-Velsky, G. y Landis, E. "An algorithm for the organization of information". *Soviet Mathematics Doklady*, 3, 1259–1263, 1962.
- Floyd, R. W. "Algorithm 245: Treesort 3". *Communications of the ACM*, 7(12), 701, 1964.
- Repositorio de `words_alpha.txt`: dwyl/english-words.
- Especificaciones del procesador: Intel Core i3-1115G4, Notebookcheck.